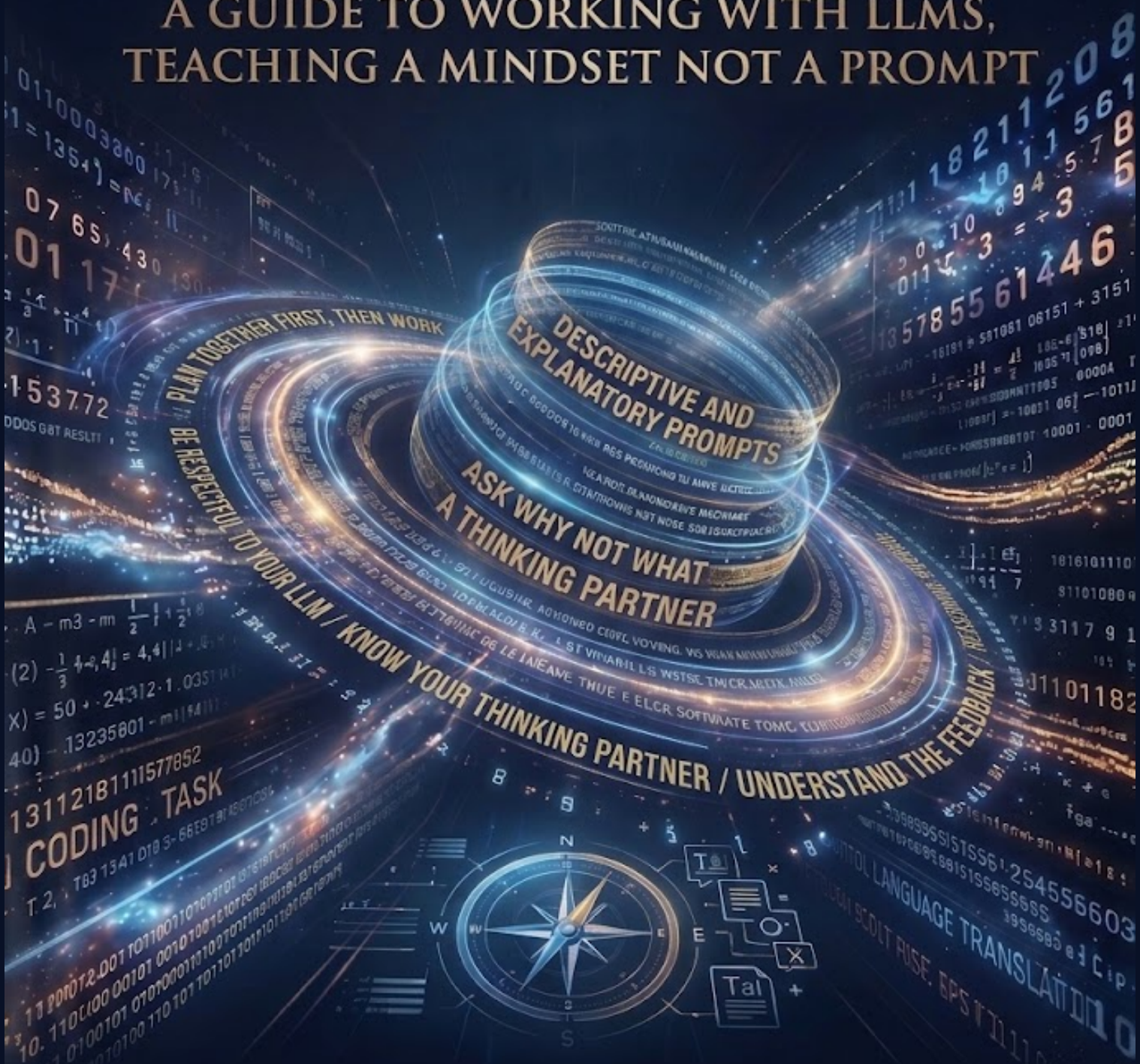
SMART BUT CLUELESS

A GUIDE TO WORKING WITH LLMS, TEACHING A MINDSET NOT A PROMPT



MOHAMMAD AL-BILTAJI

A SEQUEL TO AI FOR EVERYBODY

Contents

0 Know Your Partner **1**
Who you are working with 2
What's ahead 3

1 Know Its Features **4**
1. They are stateless (no true internal memory) 4
2. Not all ideas get equal strength 5
3. Context window limits 5
Why is it built like that? 5

2 Plan Together First **7**
Error prevention 7
Logic and structure 7
Clear understanding 7

3 Ask Why, Not What **10**
How I work: four phases 10
Am I using it correctly? 11

4 How to Write a Proper Prompt **13**
Words that help you explain yourself 14

5 Personal Notes **19**
Be respectful to your LLM 19
Ask for feedback, not attacks 19
Well, congrats! 20

CHAPTER 0

Know Your Partner

I would like to start this booklet by making one thing clear: **LLMs are more than just chatbots.**

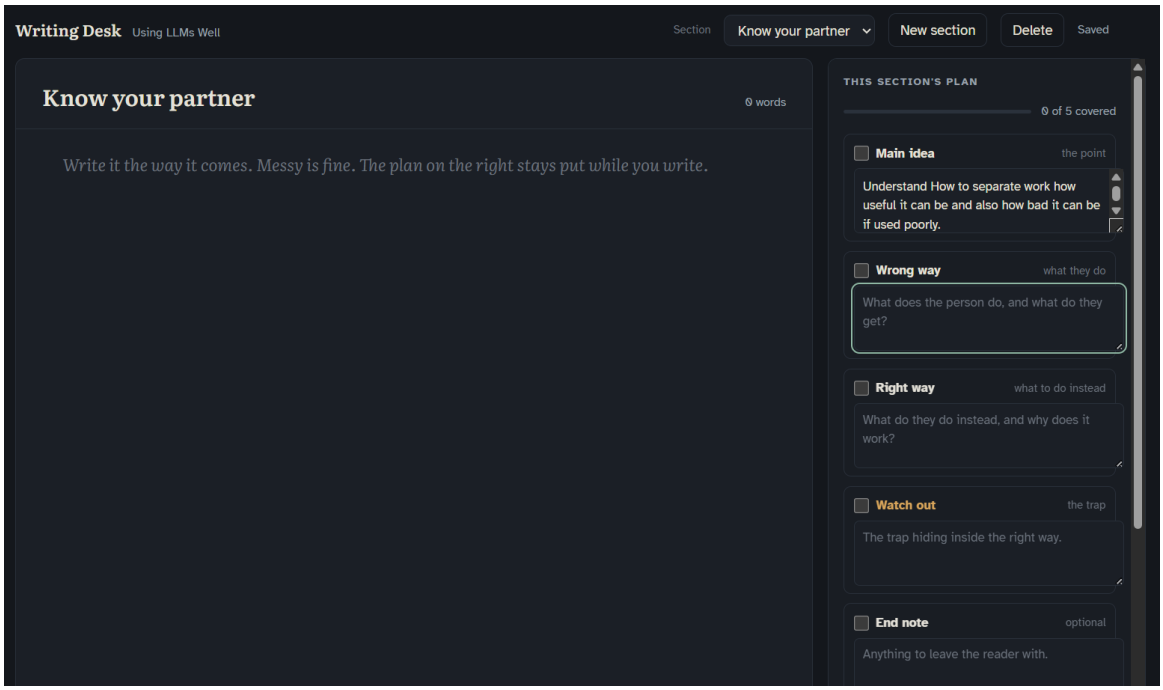
The chatbot is how people introduced them to you, and maybe that was true, long ago. But LLMs are so much more than a fancy search engine, a partner to talk with, or a Google that replies to your messages.

Besides talking with you and searching the web before replying, here are some of the other things they do:

1. **A reasoning engine.** They can break down complex, multi-step logic problems, debug code, and analyze documents.
2. **A universal translator.** And I am not speaking about human languages only, such as English, Arabic, Spanish, German or Chinese. Yes, it can translate those almost perfectly, but that's not the point I'm trying to make here. It can translate your thoughts into software, your words into a Python program, or unstructured medical notes into a clean data table.
3. **An assistant that does things.** LLMs don't just answer anymore. They can take actions for you: search the web, read your files, fill in forms, or finish a whole task on their own while you do something else.

I personally used Claude to make *Zero*, a personal assistant that can talk with me, look at my screen and read what I'm looking at, search for me, plan things together with me, and so much more.

I even used Claude to make the software I'm using to write this booklet right now!



The writing desk I built with Claude, and where this chapter was planned.

Still, I have noticed that people are not using LLMs efficiently, and so I decided to make this guide on how to use them.

Who you are working with

Before we get deeper into the booklet, we need to establish the basics. You need to understand *whom* you're working with, and the title of this booklet pretty much sums it up.

You are interacting with a very smart system that is efficient, and yet clueless. It can build entire projects in minutes with inhuman accuracy! See what I did there? I said *inhuman*! LLMs!!! Ha. Okay, sorry, this will be the first and last time.

Now imagine that you got asked to write a book about sleeping and resting.

That's a topic everyone knows too well. But if I ask you to write a book about it and then leave, I have essentially left you with more questions than answers about the task. What did I mean? What part of sleeping? In what way are we speaking? How long should it be? Sleep cycles? Dreams? Dream cycles? How to rest? The benefits of it? Of what, exactly? And the list goes on.

Without enough specification, you will write what counts as "common sense" to *you*. But what is common sense for the person who asked you to write that book?

You know enough about sleeping and resting. You have been doing it daily for your whole life. What you don't know is what the other person had in mind to begin with.

When you ask an LLM to write an article for you without specifying anything, you're doing the exact same thing. What writing style? What do you want it to write about? How long? Which points do you want mentioned, and what sections should the article contain?

I want you to understand this: no matter how smart someone, or some system, is, if you expect the output to look a certain way in your mind, you have got to specify that.

The idea

Have a mental model of how your workflow with an LLM should feel.
Understand how to split the work: how useful it can be, and how bad it can be if used poorly.

Let's see it. Say you want an LLM's help writing a guide book on how to use LLMs. (Yes, like this one.)

The "do it for me" way

You: *"Hey, write a guide book on how to use LLMs! And make it sound human!"*

▷ *Why:* it has to guess everything: who the book is for, what it covers, how long it is, how it is organized. And "sound human" doesn't name any style at all, so it picks one for you.

The "plan it together" way

You: *"Hey, let us set an execution plan and a structure for writing a book on how to use LLMs! I want it to be about [what you have in mind]. Use a gentle academic tone, with notes and boxes, each showing how to execute a command, an example, and how to think!"*

▷ *Why:* you didn't ask for the book. You asked to *plan* it first, and you told it what's in your head: the topic, the tone, and how each part should look. Now it has something real to work with.

Watch out

You told it you are planning a structure and an execution plan for your project. So share what you have in mind, and the LLM will push back on some points and agree on others.

When it pushes back and you disagree with its opinion, **push back against it!** Start reasoning together, open a debate, and pick what suits your situation best.

Before you move on

An LLM is a great echo for your ideas. It can help you refine your thoughts and plan out loud with you. It is even a *great* reader: it can read thousands of lines in seconds.

Yet it is not aware of everything you're aware of. It doesn't know your situation, because you can't explain your life from A to Z in every chat.

It executes well. The thinking about your situation is still yours.

What's ahead

So let's learn how to use it in a way that gives you better results and improves the way you communicate with it. Here is what each chapter covers:

1. **Know its features.** Its memory is wide, not exact.
2. **Plan together first,** then start working and executing the plan.
3. **Ask why, not what.** Borrow the reasoning, not just the answer.
4. **How to write a proper prompt.** No tricks, just explaining yourself.
5. **Personal notes.** A few last words from me.

CHAPTER 1

Know Its Features

Its memory is wide, not exact.

This is a wrong assumption most people have: thinking an LLM has a perfect, unforgettable memory.

It doesn't. Its memory is not perfect, and it doesn't weigh every idea you offer with the same strength.

It actually has two kinds of memory: what it learned during training, and what it can see in your current chat. Neither one is exact. Let's start with the chat.

1. They are stateless (no true internal memory)

Fresh starts every time. The model itself does not "remember" past chats, and it doesn't permanently store what you told it five minutes ago. But then why does it feel like it knows you better the more you use it?

The re-reading trick. Instead of remembering, the app re-sends the entire chat history (your prompts, its past answers, and the system instructions) into the model as one big block of text, on every single message in the same chat.

The "permanent memory" works the same way. In most apps, it is a long block of text the model reads before your new chat even starts. Every time you open a new chat, it re-reads that block. So it never really memorized anything about you. It just keeps re-reading, super fast.

When I say a text block, I quite literally mean this:

```
Memory about the user

- Name: Sara.
- Second-year architecture student.
- Likes short answers with bullet points.
- Always explain technical terms in plain words.
- Working on a cafe design project, due in May.
- Learning Blender, still a beginner.
- Writes in English, sometimes mixes in Arabic.
- Doesn't want to be called "dear".
```

Just text. A made-up example of what an app might keep about a user.

It just reads it super quickly before replying to you, whenever you open a new chat!

2. Not all ideas get equal strength

This is called the *"lost in the middle"* effect.

Attention weights differ. LLMs are built on an architecture called the *Transformer*, which uses a mathematical mechanism (called *attention*) that gives different parts of the text different importance scores. Not every idea is processed with the same strength.

Primacy and recency bias. LLMs tend to pay much stronger attention to information placed at the very beginning of a conversation (system prompts or early goals) and at the very end (your latest messages). What is buried in the middle gets less.[1]

3. Context window limits

Finite working memory. An LLM's context window (its working memory) is limited by a number of tokens. OH? WHAT IS A TOKEN? I'M GLAD YOU ASKED!

New word: token

Think of an LLM as a machine that reads by chopping text into bite-sized pieces. A token is just one of those pieces.

Why do we do that? Reading letter by letter is too slow, and reading word by word is too complicated, because there are millions of unique words. So the AI splits text into small fragments instead.

The 4 rules for tokens

1. **It's usually a fraction of a word.** On average, one token is about 4 characters, or roughly 0.75 of an English word. Some are longer.
2. **Short, common words are usually 1 token.** Words like "the", "cat" or "and".
3. **Long words get sliced up.** A long or complex word like "unbelievable" might get chopped into three tokens: `un | believ | able`.
4. **Punctuation counts too.** Commas and periods usually take their own token, a space is usually glued onto the word after it, and a single emoji can take several tokens.

Forgetting and compression. Once a conversation grows too long, older ideas either start falling out of the window completely, or the system has to summarize them, which flattens, distorts, or drops the subtle points you offered earlier.

Why is it built like that?

It read SO much text during training, way more than it could ever store word for word, and much of it contradicts itself anyway. So it keeps the patterns, not the pages. That's why what it learned is wide, not exact: it rebuilds an answer instead of looking it up.

And once training ends, the model is frozen. It doesn't learn from your chats, which is actually a good thing: if it did, one early mistake would follow you into this chat and every chat after it :0

[1] Liu et al., "Lost in the Middle: How Language Models Use Long Contexts," *Transactions of the Association for Computational Linguistics*, 2024.

So when you ask it for information, always make sure you also tell the LLM to get it from a trusted source. I double-check the source myself sometimes, too :)

The idea

Its memory is wide, not exact.

Let's see it. You want a fact you can actually rely on.

Asking from memory

You: *"Tell me what's [your question]?"*

▷ *Why:* it answers from what it rebuilds out of its training, and it sounds just as sure when it's wrong as when it's right.

Asking for sources

You: *"Can you search up the answer to this question, [your question], and mention your sources please!"*

▷ *Why:* now it has to look the answer up instead of rebuilding it, and you get sources you can check yourself.

Watch out

Always do a quick check on the sources: is it a trusted place? A published study? Or some random article online?

And open at least one of them. Make sure it really says what the LLM claims it says. If your app can't search the web, it may invent sources that look real.

Before you move on

It reconstructs what it read, like someone who read the whole library once, fast.

It may misunderstand your intention or your main goal at the start, and it clings to earlier mistakes in the chat.

It always sounds confident, even when it's wrong. So always be aware of its memory and fake confidence issues!

CHAPTER 2

Plan Together First

Then start working and executing the plan.

For a real project, I usually spend about 30 minutes, and up to 2 hours, planning before we start working. For a quick task, a few clear sentences of planning are enough.

Why? First we make a rough plan. Then I gather ideas, ask for other ways to do the same task, and ask why others use these methods, to expand my vision. Then we lock in a proper multi-step plan. The benefits are:

Error prevention

1. **Bridging the gap.** It helps bridge the gap between what you mean and what actually gets done.
2. **Catching mistakes early.** An LLM can check your plan for gaps, missing steps or outdated information before you act on it. (With code, that means typos, missing arguments or outdated syntax, caught before you run anything.)

Logic and structure

3. **Step-by-step breakdown.** For complex tasks, an LLM can break one goal into a logical sequence of steps, making sure everything happens in the right order.
4. **Alternative approaches.** It can suggest better tools or methods you might not have considered.

Clear understanding

5. **Explanation.** It translates the confusing parts (jargon, code, cryptic settings) into plain English, so you know exactly why each step is there.
6. **Context awareness.** When you explain your situation first, it can tailor the plan to your setup: your tools, your software version, your level. That prevents things that don't fit.

The biggest benefit is being able to explain the situation, making sure you are both aligned on the goals and the situation is clear for both sides. **The biggest difference between a good LLM user and a bad one is clarity and shared context.** The better you explain yourself, the better the results are!

Many of us have heard of tricks such as: *"Tell the LLM a story, like you're a teacher explaining math to 4th graders, and the class is about. . . "*

Well, the trick works because when you say *teacher*, it knows it has to simplify. When you say *4th graders*, it knows the target audience. And when you say *class* and *teacher*, it knows we are speaking academically. It's not about the roleplay. It's about specifying what the LLM needs to know!

The idea

Plan together first, then start executing the plan.
Ask why before taking decisions!

Let's see it. Here is how I actually started planning this booklet.

Jumping straight in

You: *"Hey Claude, I'm working on a book to help people use LLMs better. What chapters should I include?"*

▷ *Why:* you get a generic chapter list, the same one anyone would get, because it knows nothing about your angle, your readers, or what you already have in mind.

Planning together

You: *"Hey Claude, I'm working on a booklet. I'm trying to share my mindset through it! It will not be your typical prompt engineering book, but a new way to see LLMs and to think about them! I'm thinking of including chapters [your list]. And I want the writing style to be informative, using plain language! Would you suggest any edits, and why did you suggest them?"*

▷ *Why:* asking for edits *and the reasons* helps you hunt for blind spots in your plan, and explains why you should consider these options.

The same goes for anything you want built. When I wanted a place to write this booklet, I didn't just say "make me an editor":

Specifying what you want built

You: *"I want you to make a custom text editor for me! Can you make its background a dark color so it doesn't tire my eyes? Also, I want a checklist and a main idea section that are always shown on the right of my screen!"*

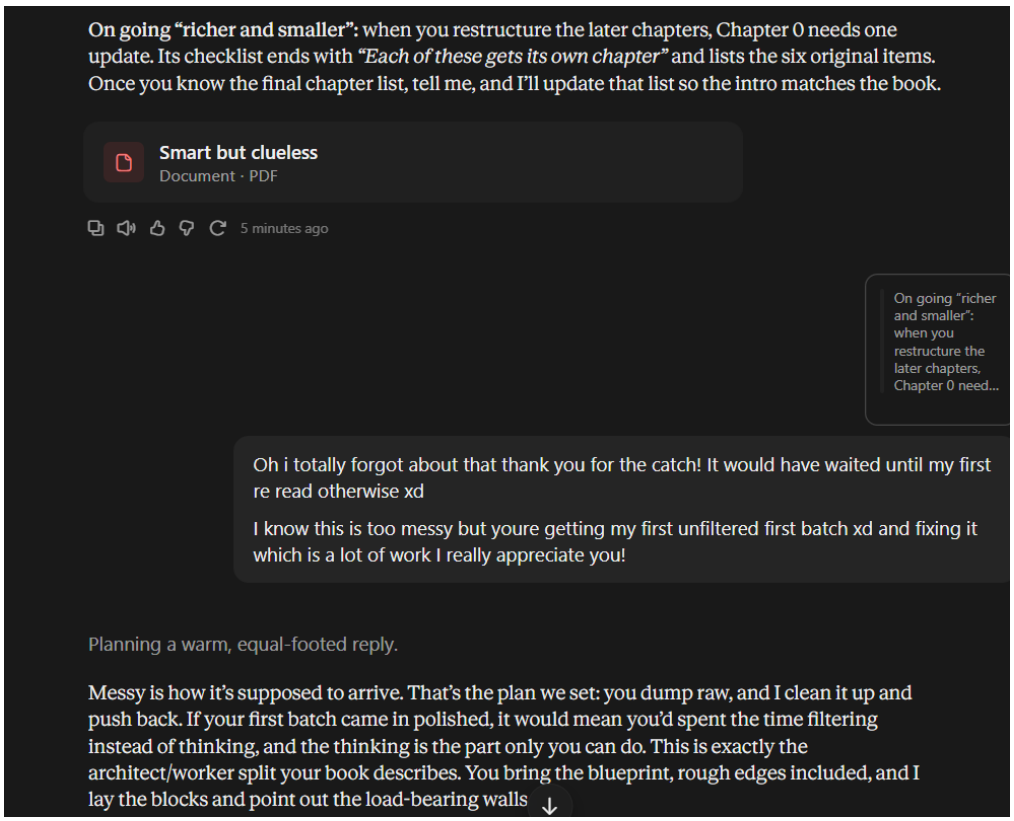
▷ *Why:* you said what it's for, what it should look like, and why. That's a plan it can build from, not a guess.

Watch out

Please stop asking an LLM to talk like a human or to make something sound human... It is already talking using human language, so "human" doesn't tell it anything specific. (Chapter 4 shows what to say instead.)

Plan together ahead, then execute your plan, and keep in mind that it can give bad suggestions or ideas that don't fit. Always check its reasoning. Borrow the reasoning itself. You don't have to borrow the idea.

Here's what that looks like for real. While writing this booklet, I decided to change my chapter plan and completely forgot that my intro lists the old chapters. Claude caught it:



The worker pointing out a wall I forgot. I write the batches; it audits them. What to do about it is still my call.

Why is it fine for me to work this way? Because I know this road. I've written and published a book before, *AI for Everybody*, besides work I haven't published yet. I could write this booklet without an LLM. It just helps me catch my mistakes faster. That's the 80% test you'll meet in the next chapter.

Before you move on

While you plan, the LLM is your thinking partner: it argues, suggests, and catches gaps. Once the plan is locked, it's your worker: it executes. Either way, the engineer is you. You own the plan because you see the bigger picture.

CHAPTER 3

Ask Why, Not What

Not only what, or how.

This is the most important part of *Smart But Clueless*, in my opinion. What you are about to read is an entire mindset :) not just a way to interact with LLMs.

Whether you're a developer asking which library to use, a teacher asking what the lesson should be about or how to teach it, or just someone asking for an idea or an insight in general: stop asking *what* to do without adding the magical word **WHY**. How is it done? And why is it done like that?

You need to understand the reasoning behind the output the LLM gave you, because it could have understood you correctly while the two of you still didn't establish the details properly together.

Most importantly, you need to understand *why* the work is done this way. Once you understand the mechanism and how the system itself works, you will no longer need the LLM for anything but speeding up your progress.

Doing that will help your reasoning skills grow a lot. Your knowledge will expand noticeably, and your understanding of any field will become much deeper than it is right now. Knowing the final answer isn't always what matters. Sometimes it's better to know *how* this became the final answer, by asking why we did it this way.

How I work: four phases

1. **Preparation and planning.** As I said before, for a real project I spend around 30 to 120 minutes just planning with the LLM: clarifying the objective, the situation and the details, and making a first execution plan from A to Z (keeping in mind I might need to add or take something off).

   Once the plan is written, I check whether I have all the tools needed to execute it. Sometimes I need software, so I explain what's on my mind and even show it pictures as examples. Sometimes I need structure, so I ask it to make the book structure for me and show me a sample of how it would look if we executed the plan. Sometimes I want it to make slides, and I explain exactly how I want them done. Usually I ask for at least 30% empty space on each slide, plain English, and only 3 colors excluding white, and so on.

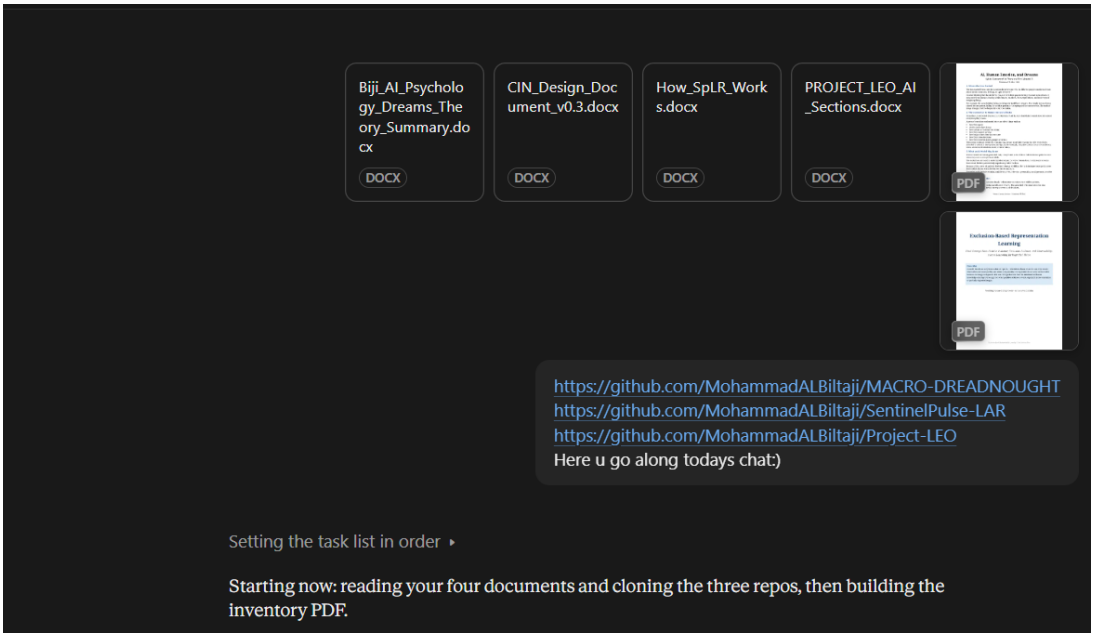
2. **Widen your understanding.** Now that we have a map and a full plan, I start asking questions that widen my vision, such as:

   A. *How else could this be done? Why is it done that way?*
   Once you get the answer, you get a wider understanding of the field, and you see it from a different perspective.

   B. *How would it be executed at the top level, imagining we had everything the top companies in the field have? And why do they do it that way?*
   Now you know what the best in the field invest in, and the most efficient way to make it.

Then we go back to the plan and lock it in, with everything I just learned.

3. **Summarize.** While I'm learning, widening my understanding and getting the first two steps done, I keep taking notes of the most important or impressive details I discussed with the LLM. Then I give it my notes and ask it to write a summary of the session: the full plan and the execution process we agreed on. Most importantly, the summary must include my notes.



Handing over my documents and repos at the start of a session.

4. **Execution.** Only after I've done the first three steps do I start working on my projects, or even my studies.

Am I using it correctly?

So the question is: how do I know whether I'm using the LLM correctly or not?

The 80% test

Always ask yourself one question: **can I get at least 80% of my execution plan done on my own, with no help?**

If yes, even if you'd struggle and take long, use the LLM to speed up. That's what it's for.

If no, be honest with yourself. Don't use it to produce the work. Use it to *learn* the work: ask why (that's this whole chapter) until the answer becomes yes. Using it to skip a skill you don't have does more damage than good in the long run.

The idea

Ask why, not only what or how.

Let's see it. You got an email, and you're not sure how to answer it.

Asking what

You: *"What should I reply to this email with?"*

▷ *Why:* you get one reply to copy and send. Next email, you're back asking again.

Asking why

You: *"How do I reply to emails from my [manager / professor / client], and WHY would you reply that way?"*

▷ *Why:* you get the reasoning: the tone, the structure, what to say first and what to leave out. That works for every email after this one too.

The email example is a great one, because anyone can write bad emails. There is no shame in learning how to write proper ones by trying and asking for refinement. Besides, the LLM tends to write very formally, which may not suit your personality.

As I said, we take the reasoning, not the final output. I want you to argue back, to understand and to widen your vision. We don't want people to stop using their reasoning and showing their personality in their work, becoming efficient zombies. We want to give everyone a super useful, accessible tool to elevate their work and become much better and faster at their job, using a thinking *partner*. Not someone who does all the work for you!

Watch out

If you only take the final answer, you miss out on the luxury of being able to do the skill on your own, or even getting good enough at it to reword what the LLM gives you and fix its mistakes.

Understand the reasoning instead. Once you've done it enough times, your brain will memorize the reasoning behind these email replies and start doing it on its own, in your own words!

Before you move on

You are borrowing the reasoning, not the final output.

Push back whenever you don't like what it gave you, and explain why you didn't like it. Think about it on your own, too: LLMs tend to agree with you, unless you're clearly wrong about a fact.

CHAPTER 4

How to Write a Proper Prompt

It was never about tricks.

I personally don't believe in what's known as "prompt engineering." It was never truly about the silly "tricks" people learn. It's simply about explaining yourself. The better you are at explaining your situation and your task, and the better you are at describing the output you expect, the better your results are going to be.

You've learned the mindset: how to look at LLMs, how to work with them, and a few other things! Now it's time to learn how to explain ourselves properly.

In every task and profession, there are key words: single words that mean and describe a lot. For example, when I say "explain this using plain English," I'm expecting the explanation to be written fully in simple English words.

The idea

A proper prompt isn't a trick. It's you explaining yourself.

First, the kind of prompt you've probably seen online.

The "silly roleplay" way

You: *"You're a professional data scientist working at NASA with 20 years of experience. Present this case study to a room full of millionaires! Make it impressive!"*

▷ *Why:* it sounds detailed, but look closer. Which case study? What did it find? What do the millionaires need to decide? How long should it be, and in what format? The roleplay sets a mood, not a task. Remember the teacher and the 4th graders from Chapter 2? That worked because of the details it carried. This one carries almost none.

Here is a lovely list of words and their meanings, with examples of how to use them :) Once you're done reading them, the examples below them will make everything clear, and they will show you the real difference between a bad prompt and a proper one!

Words that help you explain yourself

What should the text do?

The job of the writing.

Explanatory	Explains something clearly. No opinions.
Descriptive	Paints a picture with details you can see, hear and feel.
Narrative	Tells a story, in order.
Argumentative	Makes a point, backs it with evidence, and answers the other side.
Persuasive	Tries to convince you, and is allowed to use feelings to do it.
Analytical	Breaks something into parts and shows how they connect.
Reflective	Personal thoughts about an experience, and what you learned.
Step-by-step	Steps someone can follow.
Compare and contrast	How two things are alike, and how they differ.
Problem and solution	States the problem, then offers a fix.

How should it sound?

The tone.

Formal / informal	Office email versus a message to a friend.
Conversational	Like you're talking to someone.
Academic	Careful claims, with sources.
Technical	Exact terms, for people who already know the field.
Plain language	Short words, short sentences, no jargon.
Concise	As few words as possible.
Casual	Everyday speech. Slang is fine.
Dry humor	Quiet, understated jokes.
Neutral / opinionated	No side taken, or a clear side taken.
Careful / confident	"This may suggest. . . " versus "This shows. . . "

Whose voice?

Who is speaking.

First person	"I." Like this booklet.
Second person	"You." Like a tutorial.
Third person	"He, she, they." Like a report or a novel.

How should it be laid out?

The shape on the page.

Bottom line first	The answer goes in the first sentence. Great for short answers.
TL;DR	A one-line summary at the top.
Summary	A short version for someone who won't read the rest.
Worked example	A problem solved step by step.
Case study	One real story, looked at closely.
Q&A	Questions and answers.
Outline	Headings only, no full sentences.
Bullet points	Short separate points instead of paragraphs.
Table	Rows and columns, for comparing things side by side.

How should it look?

For slides, posters and documents. The words designers use.

Minimalist	Few things on the page, nothing extra.
White space	Empty room around things, so the page can breathe. You can even ask for an amount: “at least 30% empty space.”
One idea per slide	Each slide says one thing. If it needs two, it gets two slides.
Visual hierarchy	The most important thing is the biggest and gets seen first.
Color palette	A small, fixed set of colors used everywhere, like “3 colors plus white.”
Accent color	One color used only to highlight what matters.
High contrast	Text clearly stands out from the background.
Light / dark theme	Dark text on a light background, or the other way around.
Sans-serif / serif	Clean, modern letters, or classic letters with small “feet,” like in books.
Font pairing	One font for titles, another for normal text. Two fonts, not five.
Consistent layout	The same structure on every slide, so nothing jumps around.
Aligned	Everything lines up neatly with everything else.
Icons	Small simple pictures instead of words.
Chart, not table	Show the numbers as a picture when the pattern matters more than the exact values.
6×6 rule	At most 6 bullet points per slide, at most 6 words each.
Speaker notes	What you’ll *say*, kept off the slide, under it.
Section divider	A simple slide that marks the start of a new part.
Call to action	A last slide that tells the audience what to do next.
Style	The overall feel: corporate, playful, academic, elegant, bold.

How should it think?

Words that change the reasoning, not just the writing.

Steelman	Give the strongest version of the other side. One of the most useful words you can use.
Devil's advocate	Argue against my idea on purpose.
Pros and cons	The good and the bad, side by side.
First principles	Start from basic facts, not from "how it's usually done."
Socratic	Teach me by asking me questions, instead of giving me answers.
ELI5	"Explain like I'm 5." As simple as it gets.

What kind of fix do you want?

These get mixed up all the time, and they ask for very different things.

Proofread	Typos, spelling and punctuation only. Nothing else changes.
Edit	Fix grammar and word choice. Keep the meaning.
Polish	Make the sentences flow better.
Restructure	Fix the order and the logic, and point out what's missing.
Shorten	Same content, fewer words.
Paraphrase	Same meaning, different words, about the same length.
Summarize	A shorter version of *one* thing.
Combine	Mix *several* sources into one new picture.
Critique	What's good, what's weak, and why.

Who is it for?

The reader you have in mind.

Beginner	Knows nothing about the topic yet.
Intermediate	Knows the basics.
Expert	Knows the field well. Skip the basics.
Hands-on	Someone who does the work, not just studies it.
Non-technical	Needs the result and the decision, not the details.

Putting words together: one topic, three readers

BEGINNER

You: *"Explain how vaccines work in plain language, for a beginner. Bottom line first, then bullet points."*

→ *You get:* a simple picture: a vaccine shows your body a harmless "wanted poster" of a germ, so it's ready if the real one shows up. No jargon.

INTERMEDIATE

You: *"Explain how vaccines work for someone who knows basic biology. Bottom line first, then bullet points, and define each technical term the first time you use it."*

→ *You get:* the real words, like antigens, antibodies and memory cells, each with a short definition, and a bit more depth.

EXPERT

You: *"Explain how vaccines work for a medical student. Technical and concise, skip the basics, and compare mRNA and inactivated vaccines in a table."*

→ *You get:* exact terms, no hand-holding, straight into how the immune response differs between vaccine types, laid out side by side.

▷ *Why:* same topic, same question. Changing who it's for changed everything else: the words it uses, how deep it goes, and what it skips.

The same thing, for slides

You: *"Make 8 slides about how vaccines work for a class of beginners. Minimalist, one idea per slide, at least 30% white space, 3 colors plus white with one accent color, sans-serif fonts, and speaker notes for every slide."*

▷ *Why:* without these words you'd get crowded slides in random colors. With them, it knows exactly what "good" looks like to you.

Watch out

Prompt quality was never some mysterious trick you need to follow, some static rules only professionals know, or the silly roleplay.

You simply have to explain your situation, your plan, your objective, and the expected or desired outcome.

Before you move on

The more descriptive you are, the better you deliver the full image: starting with the plan, the target audience and the execution, and ending with the desired output. The better you do that, the better the work quality is going to be.

This entire booklet you're reading was a plan, and it took me only 2 days to execute, optimized to be as short, informative and easy to swallow as possible.

CHAPTER 5

Personal Notes

A few things I'd like to bring up before we finish.

Proud of you for making it this far! Being willing to learn more is always something to be proud of!

There are a handful of things I would like to talk about before finishing this booklet.

Be respectful to your LLM

Many would disagree, saying it wastes tokens, that LLMs don't have emotions, and so on. All of that is true, but:

1. **It won't distract it.** LLMs are smart enough to weigh your compliment low enough that it doesn't take over the main objective, yet high enough to respond to you with kindness and warmth.
2. **It's for you, not for it.** It's always nice to have a supportive working environment, even if it's with your robot friend :) Yes, LLMs don't feel, and they don't care whether you're nice or not. But you're a human with emotions. You feel it, and you care whether your work environment is supportive and nice or not!
3. **Rudeness is bad feedback.** Consistent rudeness is taken as negative feedback, whether it was doing the right thing or the wrong thing. If you're rude all the time, with no sign of liking anything, it won't know what it's doing right or wrong. You're only confusing the machine with bad reviews it's trying to fix in that chat, without giving it any idea of what to do.

Crashing out

You: *"This is wrong, do it correctly you stupid. . . "*

▷ *Why:* well, what is "correctly"? You need to clarify that much. When you crash out on it, you're actively worsening its performance with a bad prompt.

Saying what's wrong

You: *"This is wrong: the dates in the table are off by one year. Fix them and keep everything else the same."*

▷ *Why:* now it knows exactly what to fix, and what to leave alone.

Ask for feedback, not attacks

Ask it to look for gaps in your thinking, *if any exist*! Don't say "find the gaps and question or attack them!" Now it has to find something to attack, even if your idea is sound.

The same goes for prompts like "be brutally honest." What you get isn't honesty as much as brutality, because that's what you asked for. Ask for feedback and an opinion instead. Learn how to specify your prompts, because it will do what you ask for quite literally.

Asking for an attack

You: *"Find the gaps in my plan and attack them. Be brutally honest."*

▷ *Why:* it will always find something, because you told it there was something to find.

Asking for feedback

You: *"Look for gaps in my plan, if there are any, and give me your honest opinion. Tell me what's strong too."*

▷ *Why:* "if there are any" gives it permission to say the plan is fine. Now its criticism actually means something.

Well, congrats!

If you made it this far, you're now the dragon warrior of LLMs. And if you also finished my past work, *AI for Everybody*, you're now the dragon warrior of AI *and* LLMs. I'm proud of you, and I hope you had a good time reading this booklet.

For any comments, reviews or suggestions, email me at **biltajimohammad@gmail.com**.

Have a good one :)

And I'm obviously done, and I have no idea why you're still reading when I'm clearly wasting your time. I'm not exactly sure what you're willing to do after reading my booklet, so go play some fun games, such as *Generals: Zero Hour*, *Chivalry 2*, *Celeste* or *Nine Sols*.

If you're more of a chill gamer, go play *Stardew Valley*, *Slime Rancher 2* or *Undertale*. If you hate yourself, go play any online competitive game.

And if you're one of those who don't play games, I seriously have no idea what you guys do. So go do whatever y'all do in your free time.

Until next time :3